TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Rosa Beatriz Marcillo Loor.

PARALELO: "A"

SEMESTRE: Quinto Nivel.

ACTIVIDAD: Estudio de caso: Sistema de Control de Rescate y Adopción Animal.

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad _____

Actividad # 1

Sistema de Control de Rescate y Adopción Animal.

El rescate y la adopción animal son actividades importantes para proteger a los animales que se encuentran abandonados, perdidos, heridos o en situaciones de riesgo. Por esta razón, se propone el diseño de un Sistema de Control de Rescate y Adopción Animal que permita registrar los reportes ciudadanos, organizar los casos de rescate, asignar responsables y llevar un seguimiento de la atención veterinaria y el estado de cada animal.

El sistema contempla la gestión de diferentes especies, como perros, gatos y fauna silvestre, considerando que cada una puede requerir cuidados y procedimientos específicos. Además, permitirá administrar los expedientes de los animales, registrar su evolución, gestionar los procesos de adopción y mantener un historial de las acciones realizadas. Para desarrollar esta propuesta, se identificarán los requisitos funcionales y no funcionales, los actores involucrados y los diagramas UML, aplicando patrones de diseño y herramientas como Git y GitHub para organizar el desarrollo y facilitar el mantenimiento futuro del sistema.

Actores principales.

- Voluntario/ Veterinario: Responsable de atender el caso que se le ha asignado.
- Administrador del refugio: Supervisa los expedientes y coordina las operaciones.
- Ciudadano/ Reportante: informa una situación que requiere rescate y registra el caso.

Requerimientos funcionales.

1. Registrar casos de rescate describiendo el animal, ubicación y las condiciones que se encuentra.
2. Consultar estado de los casos de rescate que se hayan reportado.
3. Registrar la especie del animal reportado, clasificándolo como perro, gato o fauna silvestre y asociarlo con su expediente correspondiente.
4. Permitir al administrador revisar los reportes recibidos, verificar la ubicación, la descripción del animal y las condiciones antes de asignar el caso.
5. Asignar voluntarios o veterinarios para atender los casos de rescate.
6. Crear y actualizar el expediente de cada animal, incluyendo especie, descripción, estado de salud, tratamientos, historial de cambios y situación de adopción.
7. Consultar los casos de rescates asignados y actualizar su estado según las actividades realizadas.

8. Permitir al veterinario registrar las evaluaciones médicas, los tratamientos aplicados, las fechas de atención y la evolución del estado de salud del animal.
9. Enviar notificaciones a los voluntarios o veterinarios asignados cuando se les asigne un nuevo caso o se produzcan actualizaciones importantes**.**
10. Registrar las solicitudes de adopción, los datos del adoptante, la evaluación correspondiente y el resultado del proceso, incluyendo la fecha de entrega del animal cuando la adopción sea aprobada.
11. Consultar historial de los expedientes para realizar el seguimiento de los rescates, atenciones y las adopciones.
12. El sistema debe crear un expediente según la especie del animal registrado y mantener actualizado su historial de seguimiento.

Requisitos no funcionales.

1. **Escalabilidad.**
   Permitir que el sistema incorpore nuevos casos de rescate, usuarios y especies animales sin necesidad de modificar completamente su arquitectura, garantizando que pueda responder al aumento de información y solicitudes.
2. **Mantenibilidad.**
   Organizar el código en componentes con responsabilidad definidas, facilitando la corrección de errores, la actualización de funcionalidades y la incorporación de mejoras sin afectar el funcionamiento de otros módulos.
3. **Trazabilidad.**
   Conservar un historial de los cambios realizados de los expedientes de los animales, incluyendo la fecha, el responsable y la modificación efectuada. También se mantendrá un historial de versiones del código fuente mediante Git.
4. **Seguridad.**
   Controlar el acceso al sistema mediante roles y permisos para ciudadanos, voluntarios, veterinarios y administradores. Además, proteger los datos personales de los usuarios y evitar modificaciones no autorizadas en los expedientes.
5. **Disponibilidad e integridad.**
   Garantizar que la información registrada pueda consultarse cuando sea necesaria y que los datos de los rescates, tratamientos y adopciones se mantengan completos, coherentes y protegidos frente a pérdidas o modificaciones incorrectas.

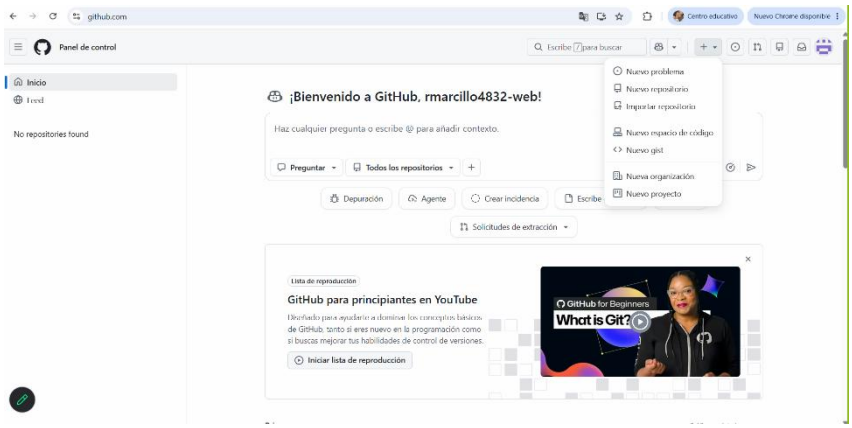
6. **Extensibilidad.**

   Facilitar la incorporación de nuevas especies, como conejos u otros animales, y de nuevas funcionalidades sin tener que modificar significativamente los componentes existentes. Para ello, se utilizarán componentes modulares y patrones de diseño adecuados.

Diseño de Sistema.

Creación del repositorio GitHub.

Se creó un repositorio en GitHub para almacenar y organizar los archivos del proyecto Sistema de Control de Rescate y Adopción Animal. Esta herramienta permite mantener un historial de cambios, respaldar el trabajo realizado y facilitar el seguimiento de las versiones del sistema. Además, se utilizará Git para gestionar los cambios y organizar el desarrollo mediante ramas, de acuerdo con el flujo de trabajo Gitflow**.**



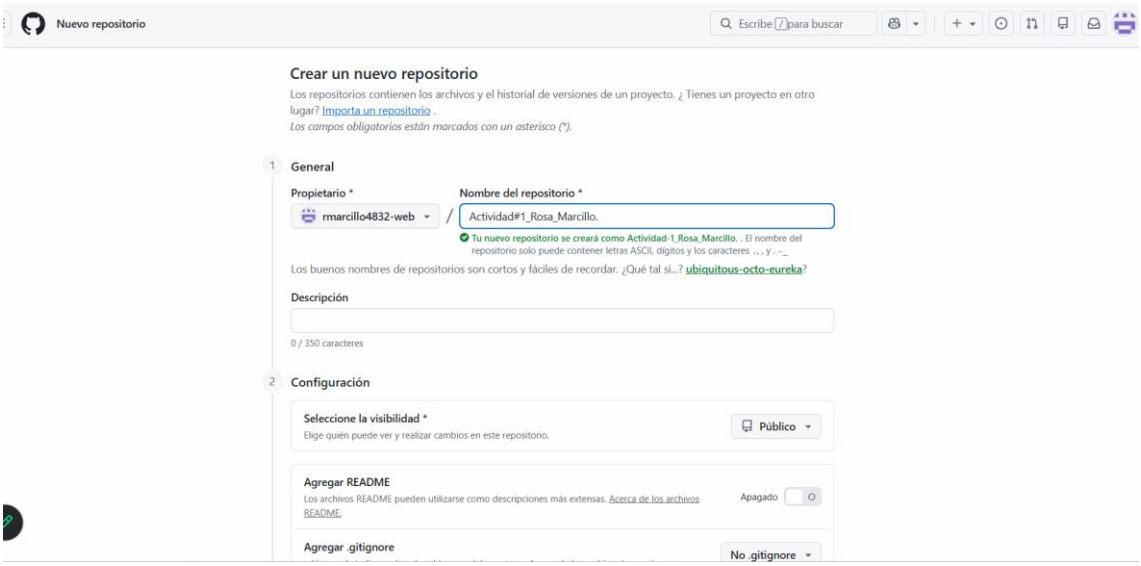


Diagrama de casos.

En este diagrama se muestran las funciones del sistema y las actividades que puede realizar cada usuario, como registrar rescates, consultar casos, atender a los animales y gestionar las adopciones. Cada actor tiene diferentes responsabilidades para ayudar a que el proceso de rescate y cuidado de los animales sea más organizado**.**

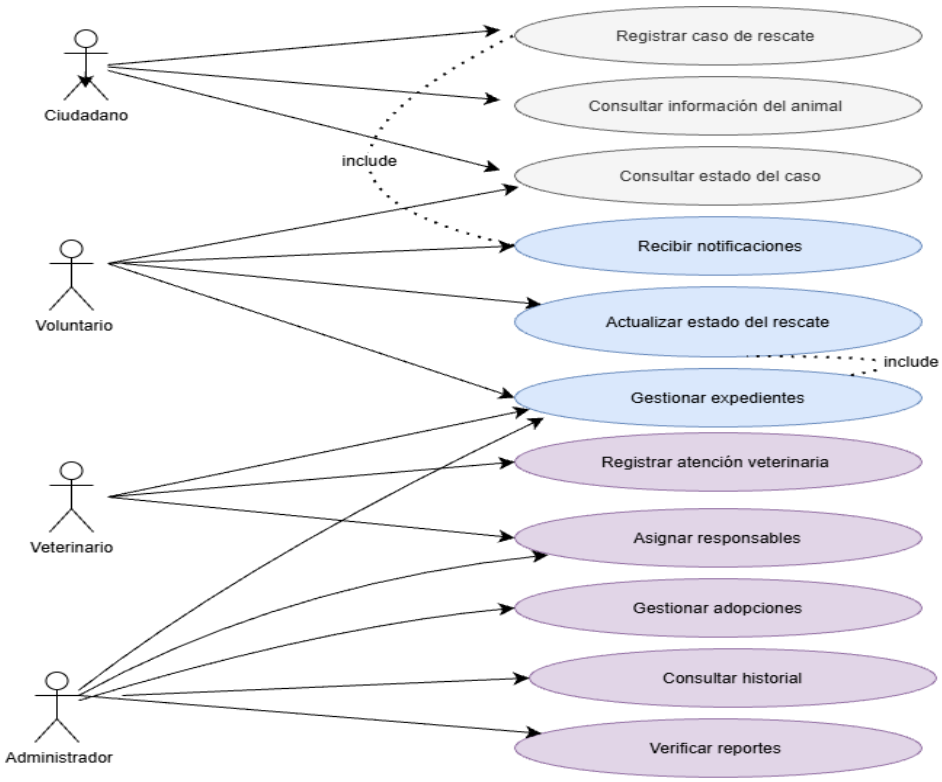
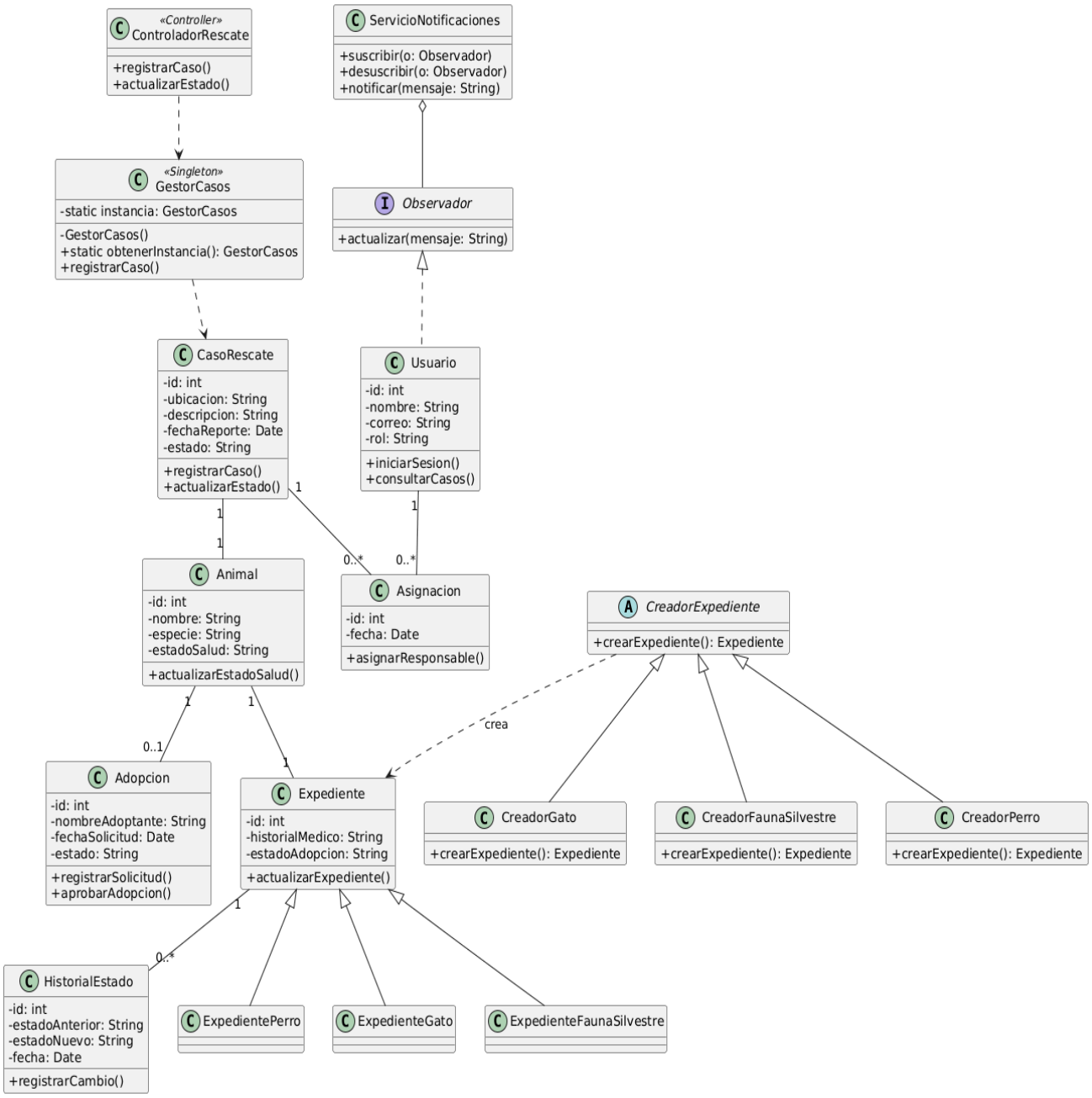


Diagrama de clases UML.



Arquitectura del diseño.

La arquitectura del sistema se organizará en diferentes capas para distribuir las funciones y facilitar el mantenimiento del software. La capa de presentación permitirá registrar los casos de rescate y consultar la información de los animales; la capa de negocio procesará los reportes, las asignaciones de responsables, la atención veterinaria y las solicitudes de adopción; y la capa de acceso a datos permitirá guardar y consultar los registros de los animales, usuarios y expedientes. La separación por capas ayuda a definir las responsabilidades de cada componente y facilita la evolución del sistema.(Cortés Ríos et al., 2022).

Para controlar las modificaciones, se utilizará Git como herramienta de control de versiones y GitHub como plataforma para almacenar el repositorio. Se propone trabajar con la rama main o master para conservar la versión estable, develop para integrar los avances y ramas feature para desarrollar funciones específicas, como el registro de rescates y la gestión de adopciones. Este flujo permite organizar los cambios y revisar las modificaciones antes de integrarlas en el proyecto.

Aplicación de patrones de diseño.

1. Patrón Singleton

Se propone utilizar el patrón Singleton en la clase GestorCasos, para que exista una sola instancia encargada de gestionar las operaciones principales de los casos de rescate. Esto permitirá centralizar los registros, mantener una mejor organización y evitar crear varias instancias del mismo gestor(Refactoring.Guru, n.d.-c).

2. **Patrón Observer**

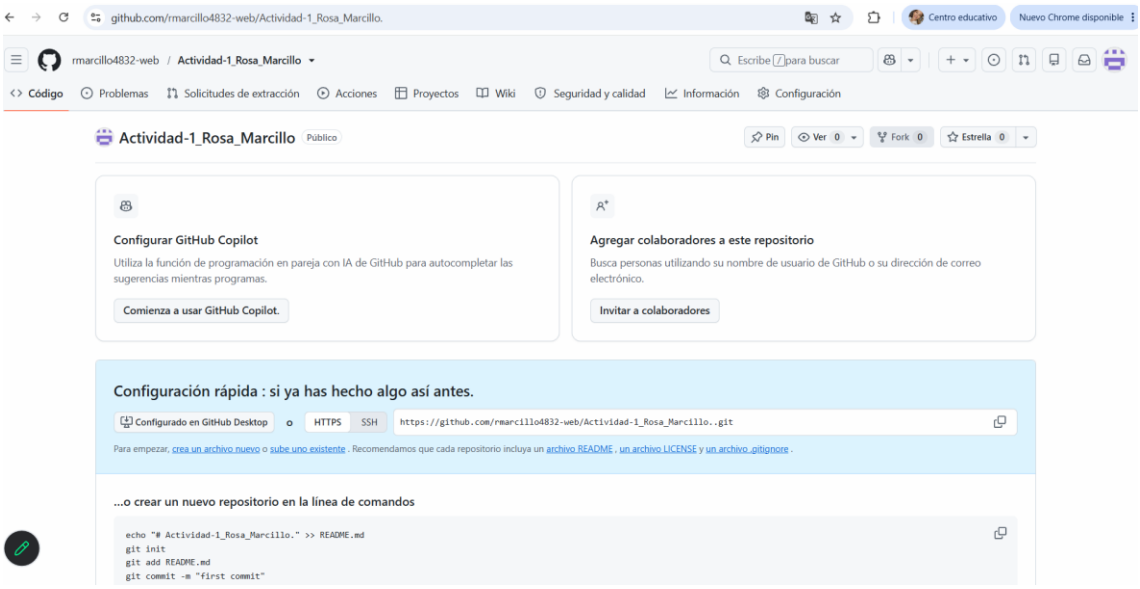
El patrón Observer permitirá notificar a los voluntarios y veterinarios cuando se registre un nuevo rescate o cambie el estado de un caso. De esta manera, los responsables podrán mantenerse informados sobre las novedades y actuar oportunamente, sin tener que revisar cada expediente manualmente (Refactoring.Guru, n.d.-b).

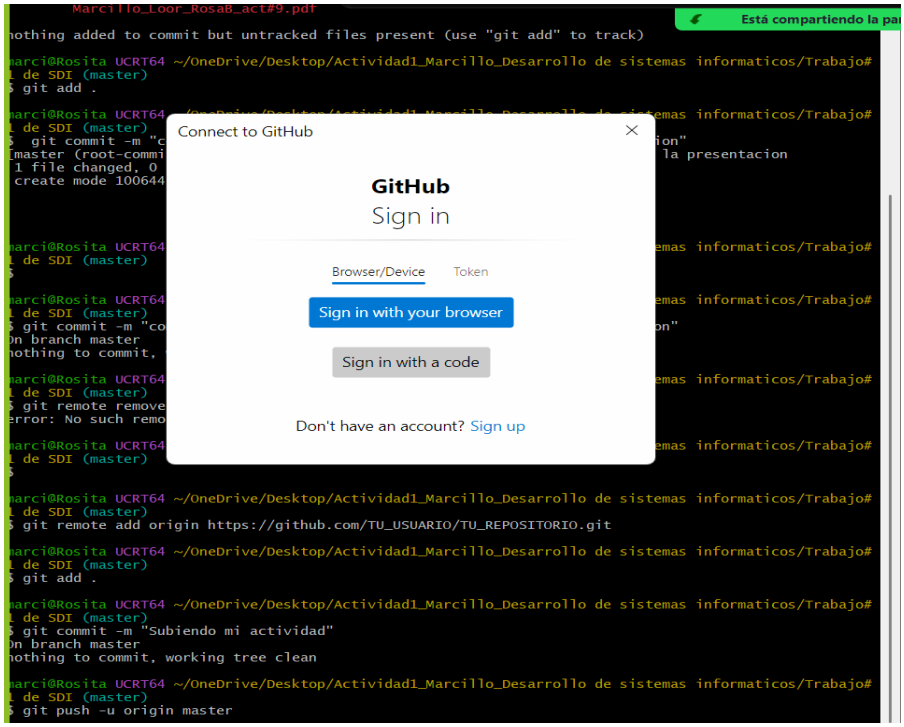
3. **Patrón Factory Method**

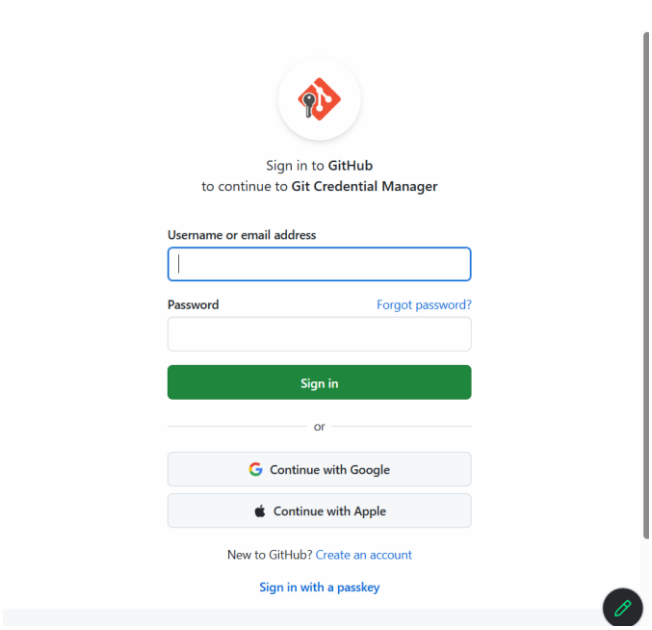
Se propone utilizar el patrón Factory Method para crear expedientes según la especie del animal rescatado, como perros, gatos o fauna silvestre. Esto permitirá registrar la información específica de cada especie y facilitar la incorporación de nuevos tipos de animales en el futuro, sin modificar toda la estructura del sistema (Refactoring.Guru, n.d.-a).

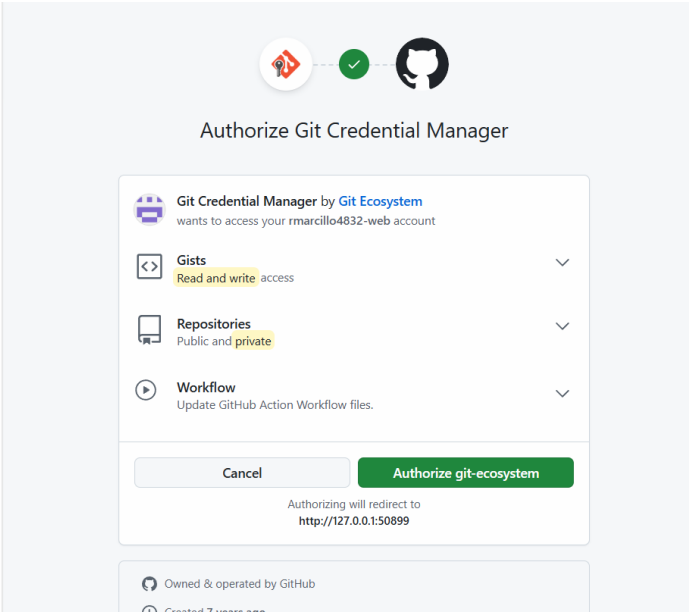
Evidencias.

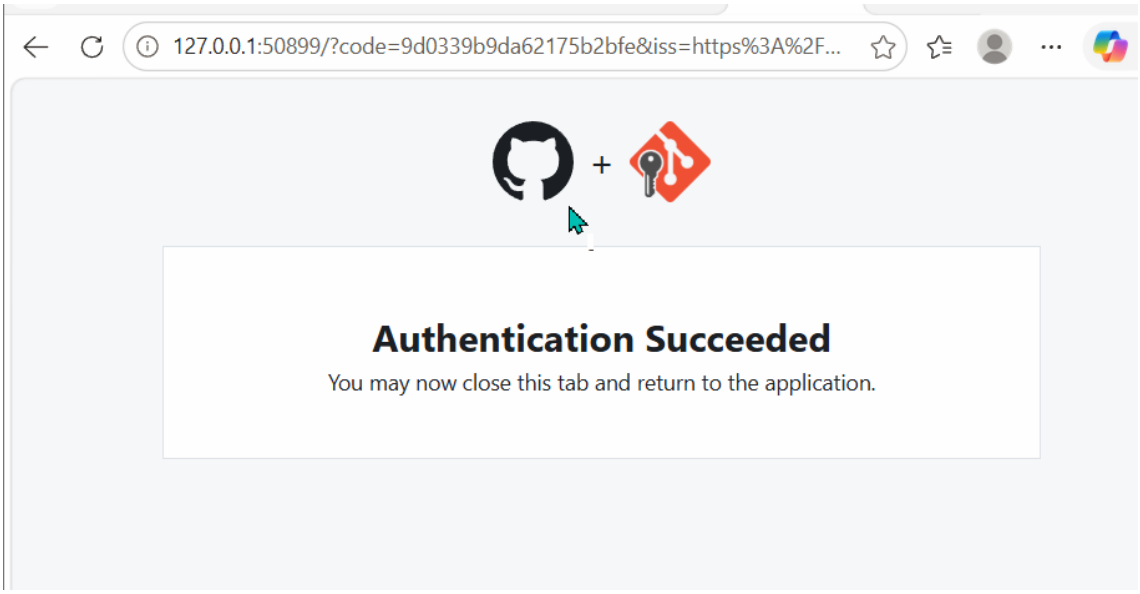
Repositorio GitHub

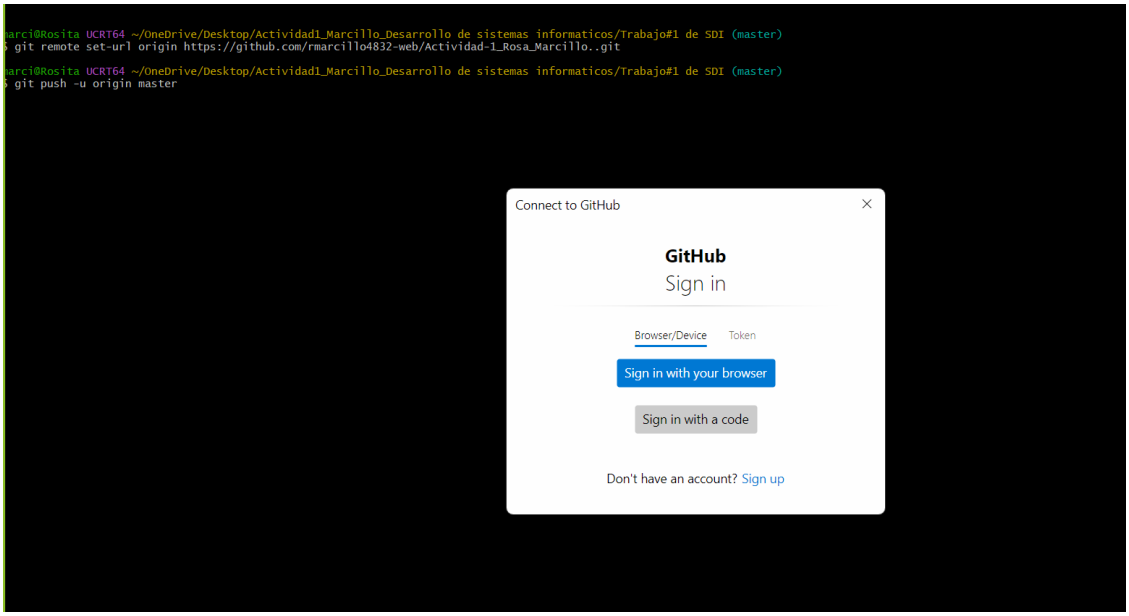


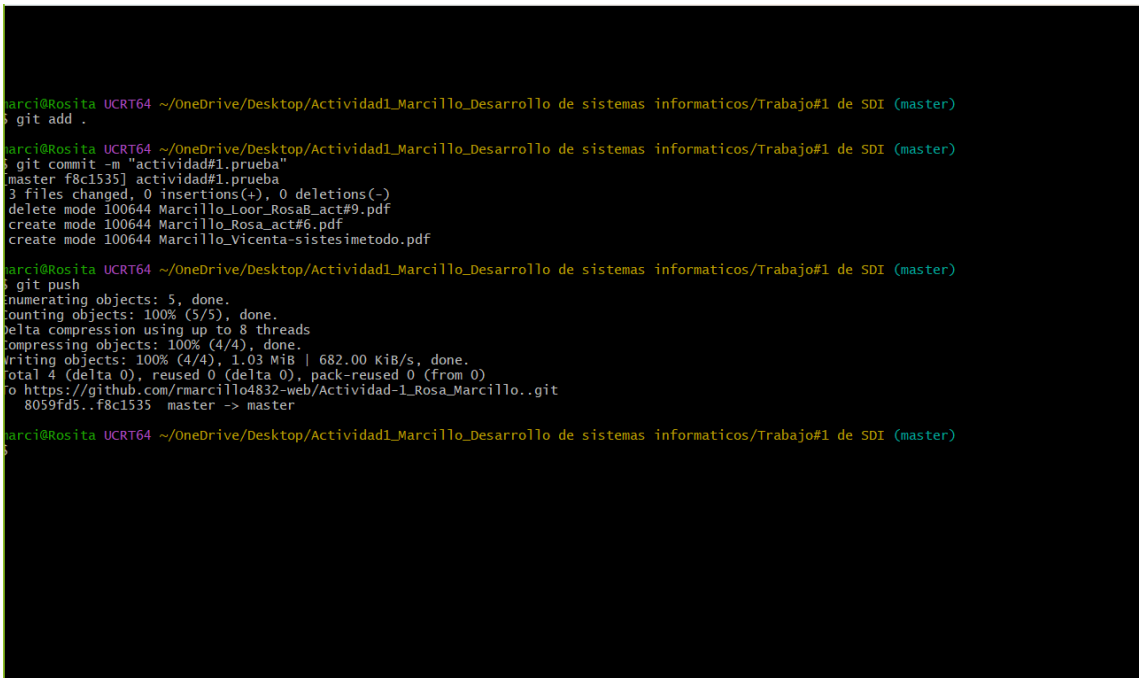



Sign in to GitHub
to continue to Git Credential Manager
Username or email address
Password
Forgot password?
Sign in
or
Continue with Google
Continue with Apple
New to GitHub? Create an account
Sign in with a passkey


Authorize Git Credential Manager
Git Credential Manager by Git Ecosystem
wants to access your rmarcillo4832-web account
Gists
Read and write access
Repositories
Public and private
Workflow
Update GitHub Action Workflow files.
Cancel
Authorize git-ecosystem
Authorizing will redirect to
http://127.0.0.1:50899
Owned & operated by GitHub


127.0.0.1:50899/?code=9d0339b9da62175b2bfe&iss=https%3A%2F...
Authentication Succeeded
You may now close this tab and return to the application.





https://github.com/rmarcillo4832-web/Actividad-1_Rosa_Marcillo.

Bibliografía

Cortés Ríos, J. C., Embury, S. M., & Eraslan, S. (2022). A unifying framework for the systematic analysis of Git workflows. Information and Software Technology, 145, 106811. https://doi.org/10.1016/j.infsof.2021.106811

Refactoring.Guru. (n.d.-a). Factory Method.

Refactoring.Guru. (n.d.-b). Observer.

Refactoring.Guru. (n.d.-c). Singleton.